Post-Quantum Credentials: Threshold ML-DSA Meets Longfellow

A short conversation deck

Changqi Sun
PhD student, Virginia Tech (advisor: Thang Hoang)

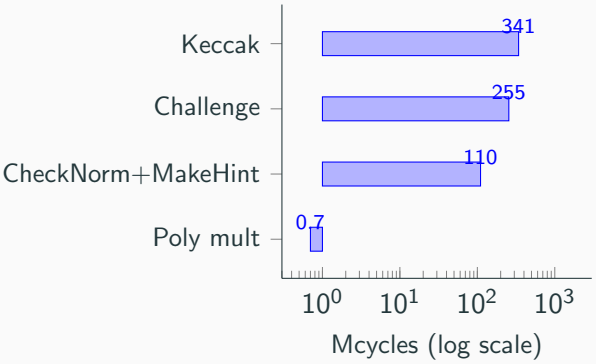
October 30, 2026

About me

- **SHIELD** (IEEE S&P 2026, co-first author): threshold ML-DSA.
  Unmodified FIPS 204 signing inside maliciously secure mixed-mode MPC;
  signatures verify under a standard verifier.
- **FLOCK** (first author, USENIX Security 2027, second round): the Falcon counterpart.
- **threshold-Falcon fixes**: reproduced the CCS 2026 artifact, found two silent bugs (PRG byte/element mix-up leaving 7/8 of the sampler buffer uninitialized; `size_t` swallowing -1). Merged upstream; signing $\approx 9\times$ faster.
- **HERO** (first author, under review at NDSS 2027): ORAM with an untrusted, stateless helper.

Common thread: prove or compute things about unmodified standard signatures.

SHIELD: where the time goes (online, ML-DSA-44)



- SPDZ for arithmetic, WRK17 garbled circuits for bit-level steps, daBits to switch.
- NTTs are linear on shares and $\hat{A}$ is public: $< 0.2\%$ of online compute.
- Boolean work dominates: Keccak-f, SampleInBall, decomposition, norm checks.
- Circuits built Verilog $\rightarrow$ Yosys $\rightarrow$ Bristol; SampleInBall 890K $\rightarrow$ 355K gates.

SHIELD cost table (signing in MPC, not verification in ZK). ML-DSA-65/87: Keccak 438/597, poly 1.21/1.37 Mcycles.

ML-DSA verification in a proof: hashing vs. lattice arithmetic

FIPS 204 Alg. 8 step	kind	Keccak-f (ML-DSA-44)
$\hat{A} \leftarrow \mathrm{ExpandA}(\rho)$ (SHAKE128)	hash	96
$tr \leftarrow H(pk)$	hash	10
μ, SampleInBall, $\tilde{c}' = H(\mu \,\|\, w_1')$	hash	≈ 10
$\mathrm{NTT}^{-1}(\hat{A} \circ \hat{z} - \hat{c} \circ \widehat{t_1 2^d})$	mod q	–
UseHint, $\|z\|_\infty < \gamma_1 - \beta$	range	–

Public key public: ≈ 10 permutations
$\approx$ 0.38M AND gates

Public key hidden (device binding, issuer hiding):
≈ 116 permutations, $\approx$ 4.5M AND gates

FIPS 204; Keccak-f[1600] = 38,400 AND (Bristol circuits); 96 + 10 count from Sharp & Grubbs, ePrint 2026/2615 §7.5.2.

What I learned reading Longfellow's ML-DSA circuit

- `tests/pq/ml_dsa`: SampleInBall, NTT-domain $Az - ct_1 2^d$, UseHint, norm checks, $\mathrm{SHAKE256}(\mu \,\|\, w_1') = \tilde{c}$, all over $\mathbb{F}_{q^6}$.
- $(\hat{A}, t_1, tr)$ are public inputs $\Rightarrow$ ExpandA and $H(pk)$ stay outside the circuit.
- ML-DSA-44 proof of possession: **858 ms** on an M4 (IETF 126).
- The community now benchmarks against it (Nowhere Man, GordieHowe, edaBitE) $\Rightarrow$ Longfellow is the reference point.

The two costs everyone points at:

1. where Keccak lives and how it is laid out (rounds per slice, batching);
2. how cheaply the hash side is linked to the $\mathbb{F}_q$ lattice side.

Possible summer project A: hidden-key ML-DSA proof of possession

Goal: device binding / issuer hiding for ML-DSA in Longfellow.

- Move ExpandA (SHAKE128 rejection sampling) and *tr* into the circuit.
- Study Keccak placement and batching; measure the cross-field link.
- Compare against the 858 ms public-key baseline.

Deliverables: circuit, cost breakdown per sub-circuit, write-up for CFRG / a paper.

Week 1

- Run `BM_MLDSA44SigProver`.
- Profile by sub-circuit: SHAKE, SampleInBall, NTT, UseHint, range checks.

Why me

- Built SHAKE / SampleInBall as Boolean circuits.
- daBits: the MPC version of the cross-field link.

Possible summer project B: an end-to-end PQ credential



- Issuer key protected by threshold signing; holder privacy by ZK. No change to the verifier or the circuit.
- Week 1: minimal ML-DSA MSO format → sign with SHIELD (KAT-checked) → prove in the existing ML-DSA test circuit.
- Honest caveat: SHIELD's offline preprocessing is heavy, so this fits low-rate issuance.

Possible summer project C: spec conformance of the ML-DSA circuit

- ECDSA circuit is proven equivalent to the spec; ML-DSA is "more difficult" (IETF 126 minutes).
- Step 1: differential and adversarial testing against FIPS 204 KATs and edge cases (hint weight ω, $\|z\|_\infty$ boundary, SampleInBall bounds).
- Step 2: a clean intermediate spec that a Lean / Verus equivalence proof can target.
- What I bring: finding silent bugs in cryptographic artifacts (threshold-Falcon). What I'd learn: the formal side, from you.

Questions for you

1. What is next for post-quantum Longfellow: issuer-side ML-DSA in mdoc, or device binding first?
2. What would make the ML-DSA circuit production-grade: speed, formal verification, or the spec?
3. Keccak in $\mathrm{GF}(2^{128})$ with a cross-field link, or native next to $\mathbb{F}_q$?
4. Where would an intern's code land: the C++ library, the Rust rewrite, or the formal side?
5. Process: PhD SWE intern pool (I'm already in it) or Student Researcher? Who would host?

Logistics: returning student (graduating June 2028); US work authorization via OPT for the summer, no sponsorship needed.

Thank you!

changqi@vt.edu changqi.dpdns.org